Customize BA Core with Codex or Claude Code

Built by BA Custom Products LLC. bacustomproducts.com

Use this workflow when the settings in Admin don't cover the change you want. You'll work in your own Git checkout, review the change, test it on a separate installation, and decide when to deploy it. For ordinary hours and price changes, use Admin or the booking settings proposal tool.

1. Start from a known version

Fork BA Core Community into your own GitHub account and clone your fork to your computer. For the current source, start your work from `upstream/main`. Replace `YOUR-ACCOUNT` with the GitHub account that owns your fork.

```
git clone https://github.com/YOUR-ACCOUNT/ba-core-community.git
cd ba-core-community
git remote add upstream https://github.com/DiyGolfGuy/ba-core-community.git
git fetch --tags upstream
git switch -c customize-booking-page upstream/main
```

To match an older installation, replace the last line with `git switch -c customize-booking-page YOUR-COMMIT-OR-TAG`. Use the source commit or matching tag from your build notes. If you have the original installer tarball, run the following in its folder on macOS, Linux, or Windows Git Bash to read its source commit.

```
gzip -dc ba-core-pi5-arm64.tar.gz | { git get-tar-commit-id; cat > /dev/null; }
```

Use a development checkout on your computer and a separate test installation. Keep the production Pi's database, config, backups, credentials, uploads and tunnel files outside your development workspace. Generate test bookings and membership records from scratch, with fictional details and placeholder values. For a payment integration test, follow the provider's sandbox instructions using dedicated test credentials stored outside Git.

2. Ask for a focused change

Open Codex or Claude Code in the development checkout. Give it the expected behavior, the feature you want to preserve, and the test you expect to pass. For example:

> Add a help link to the booking page. Make it an optional setting and preserve the page when the link is blank. Test with a fictional facility and show me the diff. Stop after local tests so I can review it.

Let the client read and modify the checkout. Review requested permissions before granting them. Run the repository's documented checks using the Node version specified by the release. Look at both the usual customer or owner screen and what happens when a setting is empty or a request fails.

3. Review the exact changes

Inspect the changes and stage only the files you intend to include.

```
git status --short
git diff
git add path/to/reviewed-file
git diff --cached
```

Check the staged files for secrets, account details, private network addresses, data snapshots, copied screenshots, test credentials and unrelated edits. Commit when the diff and tests match what you wanted.

If you share a fork or package, check the repository's license and included asset terms, and review the exact release archive separately from the source diff.

4. Back up, test, and approve the release

1. Make a full backup from BA Core Admin. Keep the archive and its passphrase separately, off the booking Pi. Record the current source commit or tag and retain the matching installation package.
2. Test the restoration procedure on an isolated Pi or test installation with synthetic records. This gives you a rollback rehearsal without using production customer data in the development checkout.
3. Build or obtain the candidate package using the release instructions. Verify its checksum and install on a separate test Pi. Check reboot recovery, the setup and Admin pages, and a fictional booking end to end. Test sandbox integrations if your change touches them.
4. Manually approve the exact reviewed package before installing it on the booking Pi. Use the upgrade or recovery procedure documented for that release. If your version only supports a fresh installation, use a separate Pi or spare drive and confirm the recovery path before switching the live hostname.
5. After installation, verify Admin and the booking site from the intended networks. Check the feature you changed and a normal booking. If you roll back, follow the documented procedure and match the prior code version with its corresponding backup.

Working with forks in GitHub